Data Structures & Algorithms

GOLDEN GATE UNIVERSITY

Lecture 1 – Introduction to Data Structures, Algorithms & Computational Thinking

Dr. Yatish Bathla,

Ph.D. (Information Technology), Obuda University, Budapest

What you will be able to do today

Explain what a data structure and an algorithm are, and why both matter

Apply the four steps of computational thinking to a small problem

Describe the properties of a correct, efficient algorithm

Count primitive operations to estimate running time

Compare growth rates: constant, log, linear, n log n, quadratic

Why study data structures?

Store

Real-world data storage

Personnel records, inventories, customer data: millions of items to keep and find.

Build

Programmer's tools

Stacks, queues and lists help build compilers, editors and games.

Model

Real-world modelling

Graphs model roads and networks; queues model waiting lines.

The right organisation of data makes a program faster, simpler, or both.

Algorithms are everywhere

Recipe

Cooking

Ordered steps turn ingredients (input) into a dish (output).

Route

Navigation

A map app searches many roads to find the shortest path.

Lookup

Library

Using the catalogue and shelf order to locate one book.

Each example has input, output and a precise sequence of steps.

Data structures vs algorithms

Data structure

A way of organising data in memory

Defines what can be stored and how it is accessed

Examples: array, linked list, stack, queue, tree, hash table, graph

Algorithm

A finite, step-by-step procedure that solves a problem

Takes input, produces output

Examples: searching, sorting, inserting, deleting

Linear and non-linear structures

Linear (this course, first)

Elements arranged one after another

Array, linked list, stack, queue

Each element has a single predecessor and successor

Non-linear (later weeks)

Elements branch or connect in many ways

Trees, heaps, graphs, hash tables

Model hierarchies and networks

Abstract data types (ADT)

What an ADT states

The operations that are available

What each operation does

Nothing about how it is stored

The implementation

Chooses the data structure behind the ADT

Decides the cost of each operation

Can change without changing user code

Four terms you will keep hearing

Term	Meaning	Example
Database	A collection of similar data	All students of a course
Record	One unit of the database	One student's entry
Field	A single item within a record	Name, ID, grade
Key	The field used to search or sort	Student ID

Computational thinking in four moves

1

Decompose

Break the problem into smaller sub-problems.

2

Find patterns

Spot what repeats or is shared between cases.

3

Abstract

Keep what matters; ignore detail.

4

Design steps

Write precise instructions: the algorithm.

Example: 'Find the tallest student in a queue' becomes 'scan once, remember the best so far'.

Algorithmic reasoning: find the maximum

```
static int findMax(int[] a) {
    int max = a[0];
    for (int i = 1; i < a.length; i++) {
        if (a[i] > max) {
            max = a[i];
        }
    }
    return max;
}
```

Key ideas

Input: a non-empty array

Idea: remember the best so far

Invariant: max is the largest of a[0..i-1]

Cost: n - 1 comparisons

Writing algorithms as pseudo-code

```
Algorithm arrayMax(A, n):
  Input: array A of n integers
  Output: largest element of A
  currentMax <- A[0]
  for i <- 1 to n - 1 do
    if A[i] > currentMax then
      currentMax <- A[i]
  return currentMax
```

Why pseudo-code?

Language independent

Precise but not tied to syntax

Easy to count operations on

Translate to Java after checking logic

Activity: think, pair, share

Problem: find whether a number appears in an unsorted list of 100 values

Think alone for 1 minute: write the steps

Pair up and compare your steps

Share: how many comparisons in the worst case?

Can you do better if the list were sorted?

What makes an algorithm good?

OK

Correct

Gives the right output for every valid input.

End

Finite

Always terminates after a limited number of steps.

Clear

Unambiguous

Each step is precisely defined.

Fast

Efficient

Uses reasonable time and memory as input grows.

Correctness comes first; efficiency decides whether it is usable at scale.

Measuring performance

Experimental study

Run the program, time it with the clock

Needs a full implementation

Depends on hardware, language and test inputs

Hard to compare two algorithms fairly

Analytical study

Count primitive operations as a function of n

Works on pseudo-code, before coding

Independent of machine

Covers all possible inputs

Counting primitive operations

```
int sum = 0;                    // once
for (int i = 0; i < n; i++) {   // ~2n control
    sum += a[i];                // n times
}
return sum;                     // once
```

Reading the count

Primitive: assignment, comparison, arithmetic, array access, return

Total: T(n) = c1 * n + c2

Exact constants depend on the machine

The shape is linear in n

Two algorithms for 1 + 2 + ... + n

Loop: add each number

sum = 0; for i = 1 to n: sum += i

About n additions

Time grows linearly: O(n)

Formula: n(n+1)/2

One multiplication, one addition, one division

Same number of steps for any n

Time is constant: O(1)

Time is not the only cost: space

Space complexity counts the extra memory an algorithm uses

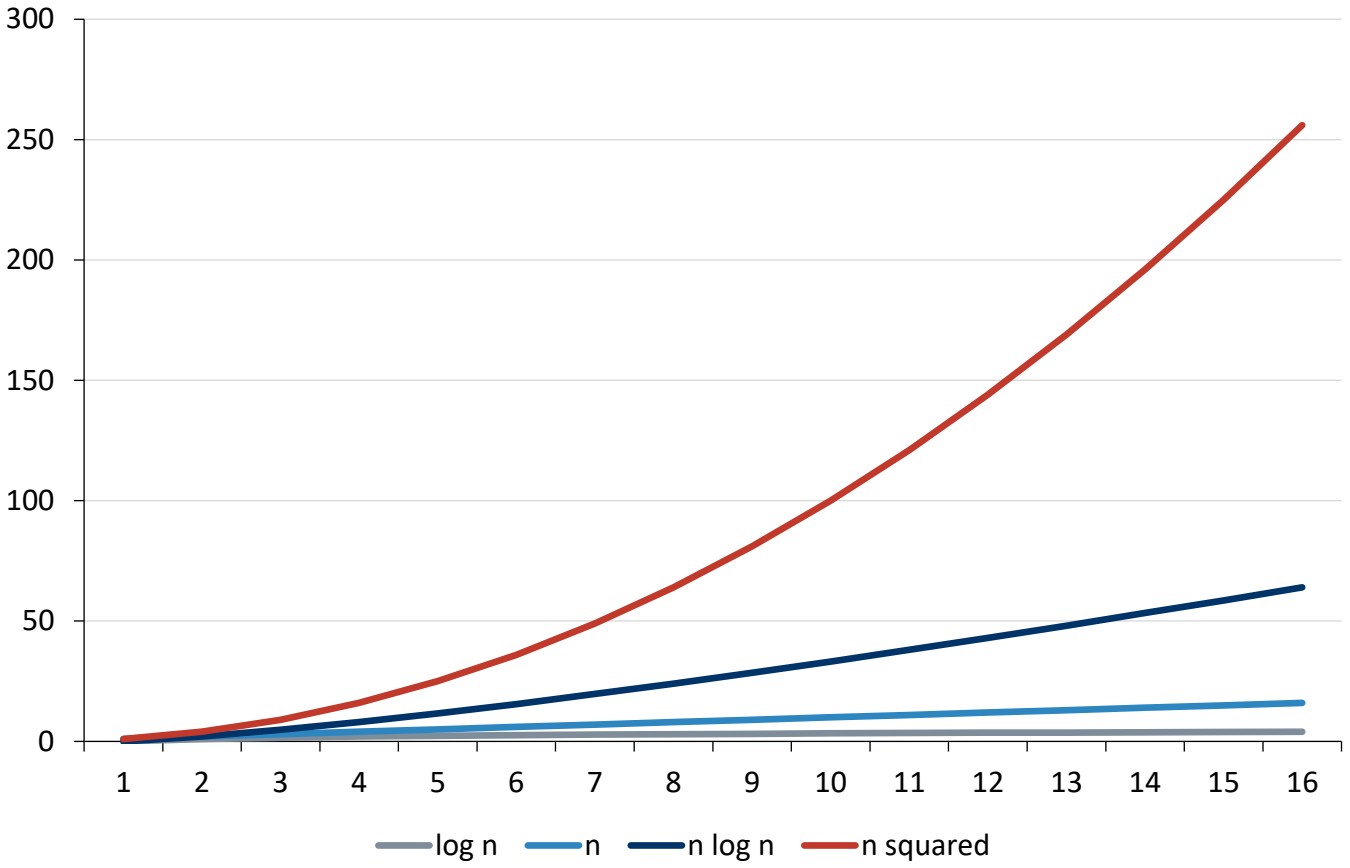
findMax uses a few variables: constant extra space

Copying an array of n items uses n extra cells

Often you can trade space for time, or time for space

This course focuses mainly on time, but always ask about memory

How fast do functions grow?



Input size n goes from 1 to 16.

log n barely moves.

n log n stays close to linear.

n squared pulls away fast.

The seven functions of algorithm analysis

Function	Name	Typical example
1	Constant	Read a[i]
log n	Logarithmic	Binary search
n	Linear	Scan an array
n log n	Linearithmic	Efficient sorting
n squared	Quadratic	Two nested loops
n cubed / 2^n	Cubic / exponential	Triple loop / all subsets

Mini quiz

1. Which is a field: a student's name, or the whole class list?
2. How many comparisons does findMax make on 500 items?
3. If time is 3n + 7, what is the growth class?
4. Which grows faster: n log n or n squared?
5. Name one advantage of analysing before coding

Recap and check for understanding

Data structure = organisation of data; algorithm = steps to solve a problem

Computational thinking: decompose, find patterns, abstract, design steps

Count operations as a function of n, not seconds on one machine

Growth rate decides scalability: log n < n < n log n < n squared

Next lecture: Big-O, Big-Omega, Big-Theta and arrays